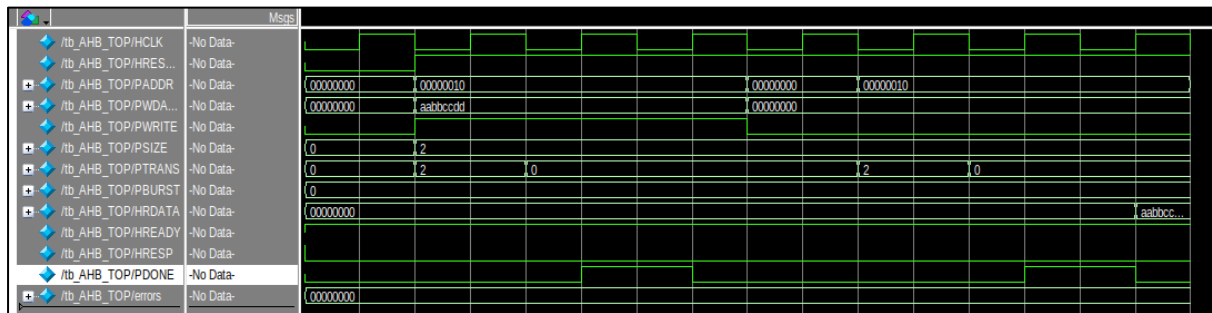
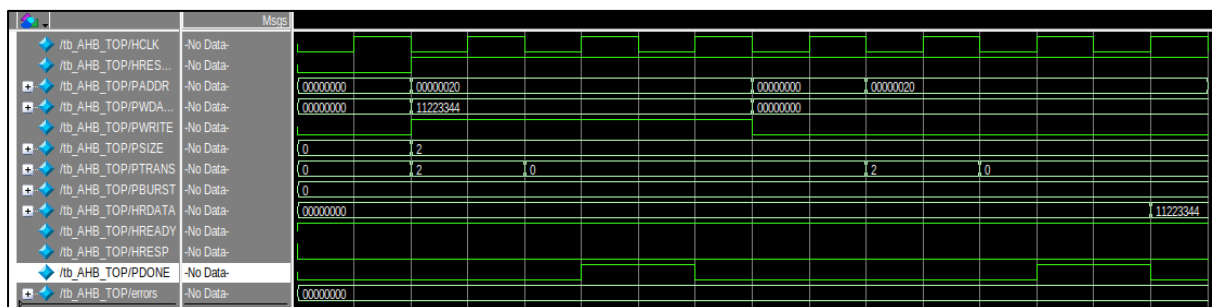


Task 2:

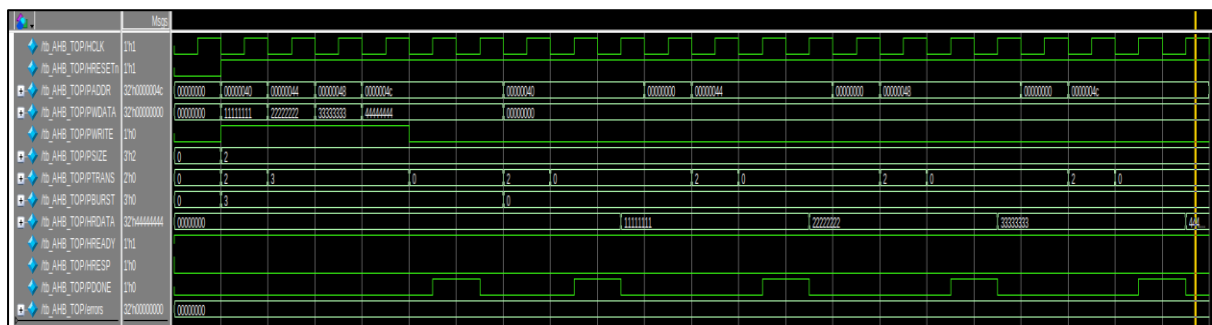
CASE 1 and 2:



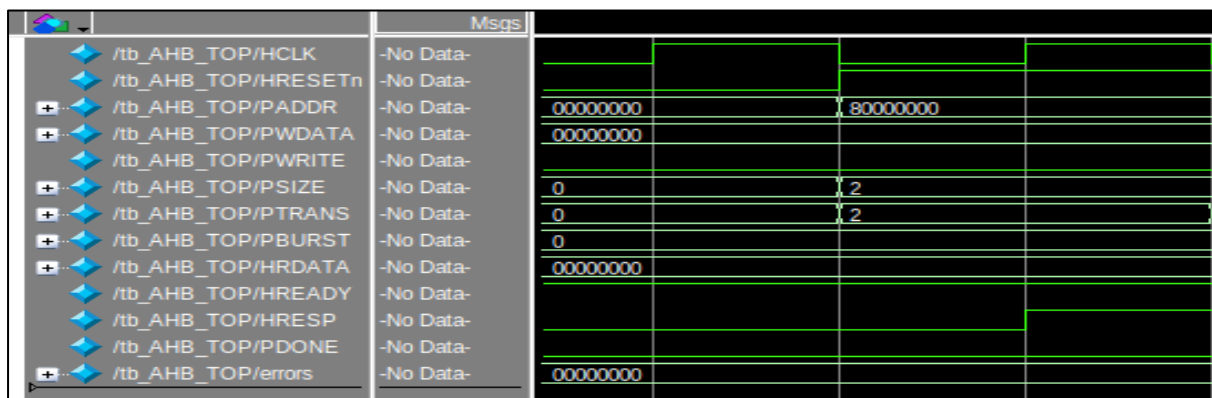
CASE 3:



CASE 4:

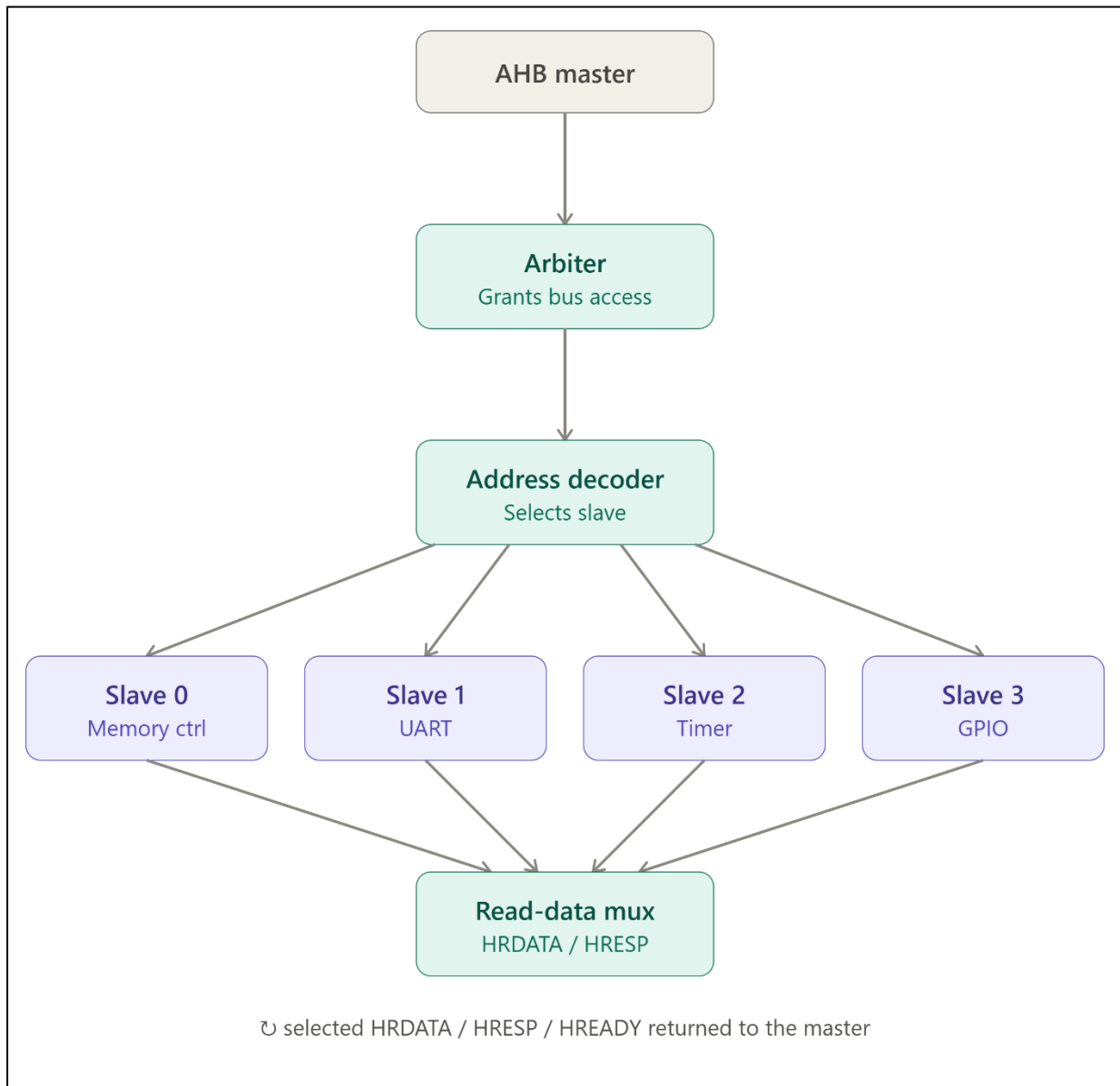


CASE 5:



TASK 3:

AHB System Block Diagram (4 Slaves)



Architecture Overview

The diagram above shows the standard AHB topology: a single master (or an arbitrated set of masters), a central arbiter, an address decoder, four slave peripherals, and a read-data multiplexer that routes the winning slave's response back to the master. This is the classic "shared bus with centralized decode and mux" structure that AMBA AHB uses instead of tri-state bus sharing.

Core components:

- **AHB master:** Initiates all transfers (address + control phase, then data phase). Typically, a CPU core, DMA controller, or debug interface. In multi-master systems, several of these connect to the arbiter, but each is functionally identical from the bus's point of view.
- **Arbiter:** Since only one master can own the bus per cycle, the arbiter samples each master's HBUSREQ line and grants ownership via HGRANT. It also drives HMASTER (identifies current owner) and can insert HSPLIT transactions for split-capable slaves.
- **Address decoder:** A combinational block that watches HADDR and generates one-hot select signals (HSEL0–HSEL3), one per slave, based on fixed or programmable address-map ranges.
- **Four slaves:** Memory controller, UART, timer, and GPIO block in this example, though any AHB-compliant peripheral works. Each slave decodes its own internal registers from HADDR but relies on the external decoder for its top-level chip select.
- **Read-data multiplexer:** Since AHB has one shared read-data bus (HRDATA), a mux (usually built from the decode signals delayed by one cycle) selects which slave's HRDATA, HRESP, and HREADY values are visible to the master.

Signal Interconnections

Signal	Direction	Purpose
HCLK	Global	Single clock driving all bus state machines
HRESETn	Global	Active-low reset
HADDR[31:0]	Master → all	Address for the current transfer
HTRANS[1:0]	Master → all	Transfer type: IDLE, BUSY, NONSEQ, SEQ
HWRITE	Master → all	1 = write, 0 = read
HSIZE[2:0]	Master → all	Byte / halfword / word / burst size
HBURST[2:0]	Master → all	Burst type (single, INCR, WRAP4, etc.)
HPROT[3:0]	Master → all	Protection attributes (privileged, cacheable, etc.)
HWDATA	Master → all	Write data (shared bus)
HSELx	Decoder → slave x	Chip-select for slave x, one per slave
HREADY (in)	Slave → mux → master	Slave signals transfer completion
HRESP[1:0]	Slave → mux → master	OKAY / ERROR / RETRY / SPLIT
HRDATA	Slave → mux → master	Read data (shared, muxed)
HBUSREQx	Master x → arbiter	Bus request from master x
HGRANTx	Arbiter → master x	Grant to master x
HMASTER	Arbiter → slaves	Identifies current bus owner (for split transactions)

Note that HADDR, HWRITE, HTRANS, etc. are broadcast to **every** slave simultaneously (unlike the old AMBA ASB's tri-state approach); only the muxed HRDATA/HRESP/HREADY are selected per-slave. This is what lets AHB decoding be simple, fast, combinational logic rather than a shared tri-state driver.

Decoding Logic

1. The decoder is purely combinational and watches only HADDR (and sometimes HTRANS to ignore IDLE cycles).
2. Each slave is assigned a fixed address range (e.g., Slave0: 0x0000_0000–0x0FFF_FFFF, Slave1: 0x4000_0000–0x4000_0FFF, etc.), often defined by the top bits of HADDR.
3. Exactly one HSELx is asserted per valid NONSEQ/SEQ transfer — this is what makes AHB decode "flat," avoiding priority-encoder ambiguity.
4. A default slave (often a dummy "error responder") handles addresses that don't decode to any real slave, preventing bus lockup on invalid accesses.
5. Because AHB is pipelined, the decoder's output for the *address phase* must be registered/delayed by one clock so the mux can correctly select data during the following *data phase* — this registered version is sometimes called HSELx_reg or handled inside the mux itself.

Operational Flow

1. **Arbitration phase:** Masters assert HBUSREQx; the arbiter picks a winner based on priority/round-robin and asserts HGRANTx and updates HMASTER.
2. **Address phase (cycle N):** The granted master drives HADDR, HTRANS=NONSEQ, HWRITE, HSIZE, HBURST. The decoder combination asserts the matching HSELx.
3. **Data phase (cycle N+1):** The selected slave either supplies HRDATA (read) or captures HWDATA (write), while driving HREADY and HRESP. AHB pipelines address and data phases, so a new address can be issued while the previous data phase completes.
4. **Muxing:** The read-data mux (using the registered select signal) routes the active slave's HRDATA/HRESP to the master.
5. **Wait states / errors:** If a slave needs more time, it drives HREADY=0, freezing the bus for one or more cycles. Errors are signalled via HRESP=ERROR, which the master must acknowledge over two cycles.
6. **Burst continuation:** For burst transfers (HBURST≠SINGLE), the master keeps HSELx asserted across the burst by issuing sequential (SEQ) transfers to consecutive addresses, avoiding redundant re-arbitration.

Task 4:

High-Speed vs. Low-Speed SoC Peripherals

SoC peripherals are typically split across two (or more) bus tiers a high bandwidth backbone (AHB/AXI) for anything that moves large or latency-sensitive data, and a low-power, low-complexity peripheral bus (APB) for simple control/status peripherals. Below are five representative examples of each.

High-Speed Peripherals (typically on AHB / AXI / high-speed matrix)

Peripheral	Typical data rate	Bus interface	Applications
DDR/LPDDR memory controller	3.2–8+ GT/s per pin (tens of GB/s aggregate)	AXI/AHB, often with a dedicated high-priority port on the interconnect	Main system memory for CPU, GPU, and DMA access
USB 3.x / USB4 controller	5–20 Gbps (SuperSpeed to SuperSpeed+)	AXI/AHB with DMA engine	External storage, docking, high-speed peripherals
PCIe controller	8–32 GT/s per lane (Gen3–Gen5), multi-lane	AXI, dedicated root-complex bridge	SSDs, NICs, accelerators, chip-to-chip links
Display controller / MIPI DSI	1.5–6+ Gbps per lane	AXI/AHB with dedicated frame-buffer DMA	Driving LCD/OLED panels, video output
Gigabit/Multi-gig Ethernet MAC	1–10+ Gbps	AXI/AHB with DMA descriptors	Networking, backhaul, automotive Ethernet
Camera interface (MIPI CSI-2)	1–4.5+ Gbps per lane	AXI/AHB with ISP DMA	Image sensors feeding an ISP pipeline
GPU / NPU accelerator	Tens to hundreds of GB/s internal	AXI (often multiple ports, QoS-aware)	Graphics rendering, ML inference

Common traits: wide data buses (64–512 bit), burst transfers, DMA-driven rather than CPU-pollled, often need QoS/priority arbitration on the interconnect, and are usually bridged onto AXI rather than plain AHB in modern SoCs because AXI supports multiple outstanding transactions and separate read/write channels.

Low-Speed Peripherals (typically on APB, bridged from AHB/AXI)

Peripheral	Typical data rate	Bus interface	Applications
UART	9.6 kbps – ~4 Mbps	APB (single cycle, no pipelining needed)	Debug console, sensor/modem communication
I²C controller	100 kbps – 3.4 Mbps (Standard to High-speed mode)	APB	EEPROM, sensors, PMIC, low-speed multi-drop buses
SPI controller	1–50 Mbps (some fast SPI up to ~100 Mbps)	APB	Flash memory, displays, sensors, ADC/DAC
GPIO controller	Not throughput-oriented; register-poll/interrupt latency in μ s range	APB	LEDs, buttons, pin muxing, interrupt-driven inputs
Timer / watchdog / RTC	N/A (register access only, μ s-level)	APB	System timekeeping, watchdog reset, scheduling ticks
PWM controller	Output frequency typically kHz range	APB	Motor control, LED dimming, audio class-D drivers
ADC/DAC control interface	Control path kbps–Mbps (data path may differ)	APB for config, sometimes AHB for streamed samples	Analog sensing, audio codecs

Common traits: narrow data buses (typically 8/16/32-bit), single, non-pipelined transfers (PENABLE/PSEL/PREADY handshake with no burst support), low gate count, low power, and bridged from the high-speed bus via an **AHB-to-APB bridge**, which appears to the AHB side as just another AHB slave while fanning out PSELx to each APB peripheral.

Why the split exists

Building every peripheral onto the high-speed AHB/AXI bus would waste silicon and power. A UART transferring at 115 kbps doesn't need a 128-bit pipelined burst-capable interface. The AHB-to-APB bridge exists precisely to let simple, latency-tolerant peripherals share a much simpler, lower-power bus protocol while still being addressable from the same overall system memory map that the high-speed bus manages.